

Candidate Transformer

Multi-Source Candidate Data Transformer — Technical Design

Transforms a recruiter CSV (structured) and a resume DOCX/PDF/TXT (unstructured) into one validated, provenance-tracked canonical candidate record, then reshapes it at runtime via a YAML projection config — no code changes.

1. Pipeline / Step Breakdown	2. Canonical Output Schema														
Manifest → Source Readers → Extraction → Normalization → Canonical Build (merge + provenance) → Projection → CLI / UI output. Readers load the CSV and resume without crashing on missing/garbage input; extractors emit traceable field candidates; normalizers standardize formats; the canonical builder merges, scores confidence and validates against JSON Schema; the projection layer reshapes the result per a runtime config.	Stable internal record, independent of any output config: candidate_id, full_name, emails[], phones[], location {city, region, country, raw}, links {linkedin, github, portfolio, other[]}, headline, years_experience, skills [{name, confidence, sources[]}], experience [], education [], provenance [], overall_confidence. The projection layer reads this schema but never mutates it — custom outputs are always derived views.														
3. Normalized Formats	4. Merge & Confidence Policy														
<table><thead><tr><th>Field</th><th>Format</th></tr></thead><tbody><tr><td>Phones</td><td>E.164, e.g. +919999999999</td></tr><tr><td>Dates</td><td>YYYY-MM</td></tr><tr><td>Country</td><td>ISO 3166-1 alpha-2, e.g. IN</td></tr><tr><td>Emails</td><td>lowercased, validated</td></tr><tr><td>URLs</td><td>https, no trailing slash</td></tr><tr><td>Skills</td><td>canonical display names (alias table), e.g. Power BI, SQL</td></tr></tbody></table>	Field	Format	Phones	E.164, e.g. +919999999999	Dates	YYYY-MM	Country	ISO 3166-1 alpha-2, e.g. IN	Emails	lowercased, validated	URLs	https, no trailing slash	Skills	canonical display names (alias table), e.g. Power BI, SQL	Match key: primary email (case-insensitive); falls back to normalized name + phone when email is absent. Conflict resolution: scalar fields take the highest-confidence value; ties favor recruiter_csv over resume. Set fields: emails, phones, links.other and skills are unioned and de-duplicated rather than overwritten. Confidence: per-field score from source reliability × extraction method (direct map > regex > LLM-parsed) × cross-source agreement; rolled up into overall_confidence.
Field	Format														
Phones	E.164, e.g. +919999999999														
Dates	YYYY-MM														
Country	ISO 3166-1 alpha-2, e.g. IN														
Emails	lowercased, validated														
URLs	https, no trailing slash														
Skills	canonical display names (alias table), e.g. Power BI, SQL														
5. Runtime Custom-Output Config	6. Edge Cases & Handling														
A YAML config remaps canonical paths to client field names, sets per-field type / normalization, and toggles provenance and confidence — independent of the canonical engine. e.g. full_name → candidate_name, location.city → current_city. on_missing: null omit error, applied per field. Output is validated against a projection-config JSON Schema before being written, so a malformed config fails fast instead of emitting silently wrong JSON.	— Missing input file → status = missing, run continues with nulls, never crashes. — Empty / unparsable file → status = empty/unreadable; no values are invented. — Scanned PDF (no text layer) → flagged unreadable with a warning; OCR is out of scope. — Conflicting scalar values → resolved by confidence/source priority and logged in provenance, not silently dropped. — Unsupported file type → rejected at the input/UI boundary before entering the pipeline.														
7. Deliberately Out of Scope (Time)	8. Run Surface														
OCR for scanned PDFs; multi-column resume layout reconstruction; database persistence; auth / user management; batch multi-candidate UI; human-in-the-loop correction screen; advanced entity resolution; external geocoding; cloud deployment and production monitoring.	CLI: python -m transformer.cli --manifest configs/sample_input_manifest.yaml --project --config configs/<name>.yaml --output data/outputs/<file>.json UI: streamlit run ui/streamlit_app.py — upload → choose projection → run → view/download JSON. Tests: pytest tests covers readers, extractors, normalizers, merge/provenance, and projection.														

Design principle: the canonical record stays stable and engine code never changes — only the runtime YAML projection varies the client-facing output.